

# Prefix Sum Interview Handbook

## 1. Algorithm Summary

**Purpose** Answer range-sum (or range-XOR/product) queries on a static array in  $O(1)$  after  $O(n)$  preprocessing, and convert  $O(n^2)$  subarray-sum counting problems into  $O(n)$ .

**Core Idea (2-3 lines)** Precompute `prefix[i] = arr[0] + arr[1] + ... + arr[i]` once. Any range sum `arr[l..r] = prefix[r] - prefix[l-1]`. For counting subarrays with sum `K`, track prefix sums seen so far in a hashmap and look up `prefix[i] - K`.

**Best Use Cases** - Multiple range-sum queries on an unchanging array - Subarray-sum-equals-K / divisible-by-K counting - 2D submatrix sum queries - Range-increment-then-read-once problems (Difference Array) - Equilibrium/pivot index, split-point problems, running-balance problems (parentheses, 0s/1s counting)

## 2. When to Use

**Conditions** - Array is static between queries (no interleaved updates) - Operation is invertible: sum, XOR, product (no zero) - Many queries expected ( $Q \sim 10^5$ ), so  $O(1)$ /query beats  $O(n)$ /query

**Input Properties** - 1D array + list of `(l, r)` queries, or - Single array + "count/find subarray(s) with sum condition", or - 2D matrix + rectangle queries `(r1, c1, r2, c2)`

**Constraints** - `n, Q` up to  $10^5 - 10^6 \rightarrow$  brute force  $O(n \cdot Q)$  will TLE, prefix sum  $O(n+Q)$  will not - Negative numbers allowed  $\rightarrow$  rules out sliding window, prefix sum + hashmap still works

**Problem Types** - Static range-sum queries - Subarray sum = `K` (count or existence) - Subarray sum divisible by `K` - Equal 0s and 1s / balanced subarrays - 2D region sum - Range update batch  $\rightarrow$  final array (Difference Array)

### Recognition Keywords (30+)

| #  | Keyword                            | Meaning                               | Use Prefix Sum?        |
|----|------------------------------------|---------------------------------------|------------------------|
| 1  | Sum of subarray                    | Need sum of <code>arr[l..r]</code>    | Yes                    |
| 2  | Range sum query                    | Multiple static queries               | Yes                    |
| 3  | Subarray sum equals <code>K</code> | Count/find subarrays = <code>K</code> | Yes                    |
| 4  | Sum divisible by <code>K</code>    | Modular subarray sum                  | Yes                    |
| 5  | Equilibrium / pivot index          | Left sum == right sum                 | Yes                    |
| 6  | Multiple queries, static array     | <code>Q</code> queries, no updates    | Yes                    |
| 7  | 2D matrix region sum               | Rectangle sum                         | Yes                    |
| 8  | Cumulative frequency               | Running count                         | Yes                    |
| 9  | Range update, read final array     | Batch increments                      | Yes (Difference Array) |
| 10 | Maximum subarray sum (plain)       | No <code>K</code> constraint          | No — Kadane's          |
| 11 | Minimum length subarray, sum       | Shrinking window                      | No — Sliding Window    |

|    |                                               |                             |                           |
|----|-----------------------------------------------|-----------------------------|---------------------------|
|    | $\geq K$ (positive)                           |                             |                           |
| 12 | Longest subarray with sum = K                 | Longest, exact sum          | Yes                       |
| 13 | Count subarrays sum = K, negatives allowed    | Non-monotonic sum           | Yes                       |
| 14 | Continuous subarray                           | Contiguous block            | Often Yes                 |
| 15 | Array updates frequently                      | Dynamic array               | No — Fenwick/Segment Tree |
| 16 | Point update + range query                    | Mixed dynamic               | No — Fenwick Tree         |
| 17 | Range update + range query                    | Both dynamic                | No — Segment Tree + Lazy  |
| 18 | Nearest greater/smaller                       | Monotonic structure         | No — Monotonic Stack      |
| 19 | Top K elements                                | Selection                   | No — Heap                 |
| 20 | Sorted array, find pair                       | Two-pointer scenario        | No — Two Pointers         |
| 21 | Connected components                          | Graph                       | No — DFS/BFS/DSU          |
| 22 | XOR of subarray                               | Range XOR                   | Yes (prefix XOR)          |
| 23 | Product of subarray (no zero)                 | Range product               | Yes                       |
| 24 | Balanced parentheses / running validity       | Prefix must stay $\geq 0$   | Yes                       |
| 25 | Equal 0s and 1s count                         | Encode 0→-1                 | Yes                       |
| 26 | Difference of counts in a range               | Two running counters        | Yes                       |
| 27 | Split array into two equal-sum halves         | Total sum compare           | Yes                       |
| 28 | Max rectangle of 1s                           | Column-wise prefix          | Yes                       |
| 29 | “Range sum query — mutable”                   | Trap: says mutable          | No — Fenwick Tree         |
| 30 | Given Q queries [l,r], answer sum each        | Classic static RSQ          | Yes                       |
| 31 | Car pooling / flight bookings / meeting rooms | Interval increments         | Yes (Difference Array)    |
| 32 | Subarray sum closest to K                     | Prefix + sort/binary search | Yes                       |

**Hidden Interview Clues** - Word “static” or “array does not change” in the problem - All queries given upfront, not interleaved with updates - Negative numbers + “subarray sum” → prefix sum + hashmap, not sliding window - “O(1) per query” in constraints - 2D grid + “submatrix sum” → 2D prefix sum - “Apply many range updates, then output final array once” → Difference Array

### 3. When NOT to Use

**Limitations** - No inverse operation for min/max, so plain prefix sum cannot answer range-min/max queries - Any single update to the source array invalidates the whole prefix array (O(n) rebuild) - Not memory-free — needs O(n) extra space (O(n·m) for 2D)

**Cases Where This Algorithm Fails** - Frequent updates interleaved with queries (array is mutable) → rebuild cost dominates - Range min/max queries → subtraction doesn’t recover min/max - k-th smallest/order-statistics in a range - Need nearest greater/smaller element

#### Better Alternatives

|  |  |
|--|--|
|  |  |
|--|--|

| Situation                           | Better Algorithm                   |
|-------------------------------------|------------------------------------|
| Point update + range sum query      | Fenwick Tree (BIT)                 |
| Range update + range query (any op) | Segment Tree with Lazy Propagation |
| Range min/max, static array         | Sparse Table                       |
| Range min/max, dynamic array        | Segment Tree                       |
| Nearest greater/smaller element     | Monotonic Stack                    |
| Top K / k-th largest                | Heap                               |
| Sorted array pair-finding           | Two Pointers                       |

## 4. Approach

**Brute Force → Optimal thinking process**

1. Brute force: for each query  $(l, r)$ , loop and sum  $\rightarrow O(n)$  per query,  $O(n \cdot Q)$  total.
2. Notice:  $\text{sum}(l, r) = \text{sum}(0, r) - \text{sum}(0, l-1)$ . Overlapping work across queries is being redone.
3. Precompute  $\text{sum}(0, i)$  for every  $i$  once  $\rightarrow O(n)$  total build.
4. Every future query becomes a single subtraction  $\rightarrow O(1)$  per query.
5. For “count subarrays with sum = K”: while scanning and building prefix sums, ask “have I seen  $\text{prefix}[i] - K$  before?” via hashmap  $\rightarrow$  turns  $O(n^2)$  into  $O(n)$ .

**How to recognize the optimization** - Whenever brute force involves **repeatedly summing overlapping ranges** of a **static** array, that repeated work is a signal to precompute. - Whenever brute force is “fix left index, extend right index, keep adding” ( $O(n^2)$ ), converting the running sum into a prefix array + hashmap lookup collapses it to  $O(n)$ .

**Why it works** - Sum is associative and has a valid inverse (subtraction), so cumulative totals can be “un-added” to isolate any subrange. - The hashmap trick works because  $\text{prefix}[r] - \text{prefix}[l-1] = K$  is just an algebraic rearrangement to  $\text{prefix}[l-1] = \text{prefix}[r] - K$  — a lookup instead of a re-scan.

## 5. Generic Template

```
def build_prefix(arr):
    n = len(arr)
    prefix = [0] * (n + 1)    # prefix[0] = 0 sentinel, avoids l=0 edge case
    for i in range(n):
        prefix[i + 1] = prefix[i] + arr[i]    # prefix[i+1] = sum of arr[0..i]
    return prefix

def range_sum(prefix, l, r):
    # returns sum of arr[l..r], 0-indexed, inclusive
    return prefix[r + 1] - prefix[l]

# --- Counting subarrays with sum == K (handles negatives) ---
def count_subarrays_with_sum_k(arr, k):
    from collections import defaultdict
    count = defaultdict(int)
    count[0] = 1    # empty prefix sum
    running = 0
    result = 0
    for x in arr:
        running += x
        result += count[running - k]    # how many earlier prefixes equal running-k
```

```
count[running] += 1
return result
```

**Variable explanation** - `prefix`: 1-indexed cumulative sum array; `prefix[i]` = sum of first `i` elements of `arr`. - `prefix[0] = 0`: sentinel so `sum(0, r)` is just `prefix[r+1]` with no special-casing. - `count` (hashmap): maps a prefix-sum value → how many times it has occurred so far. - `running`: current prefix sum while scanning left to right. - `result`: accumulates the count of valid subarrays ending at the current index.

### Minimal Competitive Programming Template

```
n = len(arr)
p = [0]*(n+1)
for i in range(n): p[i+1] = p[i] + arr[i]
# query: p[r+1]-p[l]
```

## 6. Complexity

| Case         | Build  | Query  | Notes                                  |
|--------------|--------|--------|----------------------------------------|
| Best Case    | $O(n)$ | $O(1)$ | Always $O(n)$ build regardless of data |
| Average Case | $O(n)$ | $O(1)$ | Same for all inputs                    |
| Worst Case   | $O(n)$ | $O(1)$ | No data-dependent degradation          |

**Time Complexity:**  $O(n)$  preprocessing +  $O(1)$  per query →  $O(n + Q)$  total for  $Q$  queries. For subarray-sum-K counting:  $O(n)$  single pass with hashmap. For 2D prefix sum:  $O(n \cdot m)$  build,  $O(1)$  per rectangle query.

**Space Complexity:**  $O(n)$  for the prefix array ( $O(n \cdot m)$  for 2D).  $O(n)$  extra for the hashmap variant.

## 7. Top 15 Interview Problems (Easy → Hard)

### 1. Range Sum Query — Immutable

**Difficulty:** Easy **Recognition Keywords:** “range sum query”, “immutable”, multiple queries **Why this fits:** Static array + repeated range queries = textbook prefix sum **Approach:** Build prefix array once in `__init__`; answer each query with subtraction in  $O(1)$ .

```
class NumArray:
    def __init__(self, nums):
        self.prefix = [0]*(len(nums)+1)
        for i, x in enumerate(nums):
            self.prefix[i+1] = self.prefix[i] + x

    def sumRange(self, l, r):
        return self.prefix[r+1] - self.prefix[l]
```

**Time:**  $O(n)$  build,  $O(1)$  query · **Space:**  $O(n)$

## 2. Find Pivot Index (Equilibrium Index)

**Difficulty:** Easy **Recognition Keywords:** “pivot index”, “left sum equals right sum” **Why this fits:** Need running left-sum vs total-minus-left-minus-current — a prefix-sum comparison. **Approach:** Compute total sum; scan left to right maintaining left-sum; check `left == total - left - arr[i]`.

```
def pivotIndex(nums):
    total = sum(nums)
    left = 0
    for i, x in enumerate(nums):
        if left == total - left - x:
            return i
        left += x
    return -1
```

**Time:**  $O(n)$  · **Space:**  $O(1)$

---

## 3. Running Sum of 1D Array

**Difficulty:** Easy **Recognition Keywords:** “running sum”, “cumulative” **Why this fits:** Literal definition of prefix sum. **Approach:** In-place accumulate.

```
def runningSum(nums):
    for i in range(1, len(nums)):
        nums[i] += nums[i-1]
    return nums
```

**Time:**  $O(n)$  · **Space:**  $O(1)$  extra (in-place)

---

## 4. Subarray Sum Equals K

**Difficulty:** Medium **Recognition Keywords:** “subarray sum equals K”, negatives allowed, “count subarrays” **Why this fits:** Sliding window fails with negatives; prefix sum + hashmap counts in  $O(n)$ .

**Approach:** Track prefix sums in a hashmap; for each new prefix, check how many earlier prefixes equal `prefix - K`.

```
from collections import defaultdict
def subarraySum(nums, k):
    count = defaultdict(int)
    count[0] = 1
    running = res = 0
    for x in nums:
        running += x
        res += count[running - k]
        count[running] += 1
    return res
```

**Time:**  $O(n)$  · **Space:**  $O(n)$

---

## 5. Continuous Subarray Sum (Divisible by K)

**Difficulty:** Medium **Recognition Keywords:** “divisible by K”, “continuous subarray”, “sum multiple of K”

**Why this fits:** Two prefix sums with the same remainder mod K mean the subarray between them is divisible by K. **Approach:** Track `prefix % k` → earliest index seen; if same remainder reappears  $\geq 2$  indices

apart, return True.

```
def checkSubarraySum(nums, k):
    seen = {0: -1}
    running = 0
    for i, x in enumerate(nums):
        running += x
        rem = running % k
        if rem in seen:
            if i - seen[rem] > 1:
                return True
        else:
            seen[rem] = i
    return False
```

**Time:**  $O(n)$  · **Space:**  $O(\min(n,k))$

---

## 6. Product of Array Except Self

**Difficulty:** Medium **Recognition Keywords:** “except self”, “prefix and suffix product” **Why this fits:** Prefix-product from the left combined with suffix-product from the right gives each answer without division. **Approach:** Left pass fills prefix products; right pass multiplies in suffix products.

```
def productExceptSelf(nums):
    n = len(nums)
    res = [1]*n
    left = 1
    for i in range(n):
        res[i] = left
        left *= nums[i]
    right = 1
    for i in range(n-1, -1, -1):
        res[i] *= right
        right *= nums[i]
    return res
```

**Time:**  $O(n)$  · **Space:**  $O(1)$  extra (excluding output)

---

## 7. Contiguous Array (Equal 0s and 1s)

**Difficulty:** Medium **Recognition Keywords:** “equal number of 0s and 1s”, “longest subarray” **Why this fits:** Encode 0 as -1; longest subarray with sum 0 = longest balanced subarray. Classic prefix-sum-first-occurrence trick. **Approach:** Track first index each prefix sum occurs; max gap between repeats is the answer.

```
def findMaxLength(nums):
    seen = {0: -1}
    running = 0
    best = 0
    for i, x in enumerate(nums):
        running += 1 if x == 1 else -1
        if running in seen:
            best = max(best, i - seen[running])
        else:
            seen[running] = i
    return best
```

**Time:**  $O(n)$  · **Space:**  $O(n)$

---

## 8. Range Sum Query 2D — Immutable

**Difficulty:** Medium **Recognition Keywords:** “2D matrix”, “submatrix sum”, “region sum” **Why this fits:** 2D generalization of prefix sum using inclusion-exclusion. **Approach:** Build `prefix[i][j]` = sum of rectangle `(0,0)` to `(i-1,j-1)`; answer via inclusion-exclusion.

```
class NumMatrix:
    def __init__(self, matrix):
        rows, cols = len(matrix), len(matrix[0])
        self.p = [[0]*(cols+1) for _ in range(rows+1)]
        for i in range(rows):
            for j in range(cols):
                self.p[i+1][j+1] = (matrix[i][j] + self.p[i][j+1]
                                     + self.p[i+1][j] - self.p[i][j])

    def sumRegion(self, r1, c1, r2, c2):
        p = self.p
        return (p[r2+1][c2+1] - p[r1][c2+1] - p[r2+1][c1] + p[r1][c1])
```

**Time:**  $O(n \cdot m)$  build,  $O(1)$  query · **Space:**  $O(n \cdot m)$

---

## 9. Corporate Flight Bookings (Difference Array)

**Difficulty:** Medium **Recognition Keywords:** “range increment”, “apply to bookings[i..j]”, “final array” **Why this fits:** Many range-add updates, array read once at the end — the reverse (difference-array) form of prefix sum. **Approach:** Mark `+val` at start, `-val` just after end; take prefix sum of the diff array to get final values.

```
def corpFlightBookings(bookings, n):
    diff = [0]*(n+1)
    for first, last, seats in bookings:
        diff[first-1] += seats
        diff[last] -= seats
    for i in range(1, n):
        diff[i] += diff[i-1]
    return diff[:n]
```

**Time:**  $O(n + b)$  · **Space:**  $O(n)$

---

## 10. Subarray Sums Divisible by K (Count, Not Boolean)

**Difficulty:** Medium **Recognition Keywords:** “count subarrays”, “sum divisible by K”, negatives allowed **Why this fits:** Same remainder-bucket idea as #5, but counting combinations via `C(count, 2)` style accumulation. **Approach:** Track frequency of each prefix-sum remainder; every pair sharing a remainder forms a valid subarray.

```
def subarraysDivByK(nums, k):
    from collections import defaultdict
    count = defaultdict(int)
    count[0] = 1
    running = res = 0
    for x in nums:
```

```

        running = (running + x) % k
        res += count[running]
        count[running] += 1
    return res

```

**Time:**  $O(n)$  · **Space:**  $O(k)$

## 11. Maximum Size Subarray Sum Equals K

**Difficulty:** Medium **Recognition Keywords:** “maximum length”, “sum equals K”, negatives allowed **Why this fits:** Need the *longest* subarray summing to K — store first occurrence only (not count) of each prefix sum. **Approach:** Hashmap stores earliest index of each prefix sum; check `prefix - k` for max length.

```

def maxSubArrayLen(nums, k):
    seen = {0: -1}
    running = best = 0
    for i, x in enumerate(nums):
        running += x
        if running - k in seen:
            best = max(best, i - seen[running - k])
        if running not in seen:
            seen[running] = i
    return best

```

**Time:**  $O(n)$  · **Space:**  $O(n)$

## 12. Number of Ways to Split Array into Three Sum-Equal Parts / Equal Sum Array Splits

**Difficulty:** Medium-Hard **Recognition Keywords:** “split array into parts”, “equal sum”, “waysToSplit” **Why this fits:** Total sum must be divisible; prefix sums identify valid split points in  $O(n)$  instead of trying all pairs. **Approach:** Compute prefix sums; target = total/3 (or /2); count how many prefix points equal target, 2xtarget, etc., using two pointers or hashset on prefix values.

```

def waysToSplit(nums):
    n = len(nums)
    prefix = [0] * (n+1)
    for i, x in enumerate(nums):
        prefix[i+1] = prefix[i] + x
    total = prefix[n]
    MOD = 10**9 + 7
    res = 0
    j = k = 0
    for i in range(1, n-1):
        left = prefix[i]
        if left * 3 > total:
            break
        j = max(j, i+1)
        while j < n-1 and prefix[j] < 2*left:
            j += 1
        k = max(k, j)
        while k < n-1 and prefix[k] <= total - left:
            k += 1
        res = (res + (k - j)) % MOD
    return res

```

**Time:**  $O(n)$  · **Space:**  $O(n)$



---

### 13. Count Number of Nice Subarrays (Exactly K Odd Numbers)

**Difficulty:** Hard **Recognition Keywords:** “exactly K odd numbers”, “count subarrays” **Why this fits:**

Reframe “odd count” as a prefix sum of 1s/0s (odd→1, even→0); reduces to Subarray Sum Equals K.

**Approach:** Convert to binary array of parities; apply the same prefix-sum + hashmap counting as problem #4.

```
def numberOfSubarrays(nums, k):
    from collections import defaultdict
    count = defaultdict(int)
    count[0] = 1
    running = res = 0
    for x in nums:
        running += x % 2
        res += count[running - k]
        count[running] += 1
    return res
```

**Time:**  $O(n)$  · **Space:**  $O(n)$

---

### 14. Maximum Sum of Two Non-Overlapping Subarrays

**Difficulty:** Hard **Recognition Keywords:** “two non-overlapping subarrays”, “fixed lengths L and M”, “max sum” **Why this fits:** Fixed-length window sums via prefix sum, then combine best-left / best-right windows

around a split. **Approach:** Build prefix sums; precompute best L-length window ending by index and best M-length window; slide the split point.

```
def maxSumTwoNoOverlap(nums, L, M):
    n = len(nums)
    prefix = [0] * (n+1)
    for i, x in enumerate(nums):
        prefix[i+1] = prefix[i] + x

    def best(l, m):
        best_l = 0
        res = 0
        for i in range(l+m, n+1):
            best_l = max(best_l, prefix[i-m] - prefix[i-m-1])
            res = max(res, best_l + prefix[i] - prefix[i-m])
        return res

    return max(best(L, M), best(M, L))
```

**Time:**  $O(n)$  · **Space:**  $O(n)$

---

### 15. Count Subarrays With Sum Closest to a Target / Range Sum Query with Constraints (Sorted Prefix + Binary Search)

**Difficulty:** Hard **Recognition Keywords:** “sum closest to K”, “range sum bounded between lower and upper”

**Why this fits:** After computing prefix sums, finding a subarray whose sum lies in `[lower, upper]` becomes: for each prefix, binary search how many earlier (sorted) prefixes fall in `[prefix-upper, prefix-lower]` — needs a sorted/BIT structure over prefix values. **Approach:** Use a Binary Indexed Tree (or sorted list + bisect) over prefix sums; for each new prefix, query count of earlier prefixes in range, then insert.

```

import bisect
def countRangeSum(nums, lower, upper):
    prefix = 0
    sums = [0]
    res = 0
    sorted_prefixes = [0]
    for x in nums:
        prefix += x
        lo = bisect.bisect_left(sorted_prefixes, prefix - upper)
        hi = bisect.bisect_right(sorted_prefixes, prefix - lower)
        res += hi - lo
        bisect.insort(sorted_prefixes, prefix)
    return res

```

**Time:**  $O(n \log n)$  · **Space:**  $O(n)$

## Quick Recap Table

| #  | Problem                              | Core Trick                              |
|----|--------------------------------------|-----------------------------------------|
| 1  | Range Sum Query — Immutable          | $\text{prefix}[r+1] - \text{prefix}[l]$ |
| 2  | Find Pivot Index                     | left vs total-left-cur                  |
| 3  | Running Sum of 1D Array              | in-place accumulate                     |
| 4  | Subarray Sum Equals K                | prefix + hashmap count                  |
| 5  | Continuous Subarray Sum              | $\text{prefix} \% k$ repeat             |
| 6  | Product of Array Except Self         | prefix & suffix product                 |
| 7  | Contiguous Array                     | $0 \rightarrow -1$ , prefix sum = 0 gap |
| 8  | Range Sum Query 2D                   | inclusion-exclusion                     |
| 9  | Corporate Flight Bookings            | difference array                        |
| 10 | Subarray Sums Divisible by K         | remainder frequency                     |
| 11 | Max Size Subarray Sum = K            | first-occurrence hashmap                |
| 12 | Ways to Split Array                  | prefix + two pointers                   |
| 13 | Count Nice Subarrays                 | parity as prefix sum                    |
| 14 | Max Sum of Two Non-Overlap Subarrays | fixed-window prefix                     |
| 15 | Count Range Sum                      | prefix + sorted/BIT                     |